

Final Report for Syslab
Mini Color NN: A Complete Color Quantization
of CIFAR-10 Dataset for CNN Image Classification
Sophia Huang
May 21, 2025
Yilmaz - Period 1

Table of Contents

[

Abstract

I. Introduction

II. Background

III. Applications

IV. Methods

V. Results

VI. Limitations

VII. Conclusion

VIII. Future Work and Recommendations

References

Appendix

]

Abstract

[...]

[

The following template is the minimum information I would like you to include in your report and I have included them only to give you some idea about the expected content.

Please don't just provide inline answers. If you do, you will lose a lot of points.

]

I. Introduction

Convolutional Neural Networks (CNNs) have significantly advanced the field of image classification, enabling major progress in areas such as medical imaging, autonomous driving, and security surveillance. Despite their impressive accuracy and versatility, CNNs are often computationally intensive and memory-demanding. These requirements present a major obstacle to deploying such models on resource-constrained devices, including embedded systems, surveillance cameras, and Internet of Things (IoT) hardware. These platforms typically have limited processing power, memory, and energy resources, which makes it difficult to support real time image recognition using standard high resolution or high precision inputs.

This project was motivated by the need to address the inefficiency of using full-resolution color image data in low-resource environments. While previous work has explored techniques such as grayscale conversion, binary encoding, and random noise quantization to reduce image size, these methods often result in significant information loss, which can degrade the performance of machine learning models. The goal is to find a better balance between visual information preservation and computational efficiency.

I propose a novel method that combines Floyd-Steinberg dithering and vector quantization using k-means clustering, applied to the full-color CIFAR-10 dataset. Floyd-Steinberg dithering diffuses the quantization error to neighboring pixels, preserving edge detail and visual structure, even with a limited color palette. Meanwhile, k-means clustering compresses the color space by mapping each pixel to one of a small number of representative colors. When used together, these techniques produce visually coherent images that retain enough meaningful information for CNNs to learn from, while significantly reducing the input data complexity.

This approach differs from prior work, which often focuses on grayscale or monochrome datasets, and rarely considers full-color compression combined with error diffusion techniques. To my knowledge, this is the first attempt to apply both Floyd-Steinberg dithering and vector quantization to a full-color benchmark dataset in order to support lightweight neural network applications.

The result is the Mini-Color dataset, a highly compressed yet visually informative version of CIFAR-10. This dataset is released under an open license to encourage further research and experimentation in efficient deep learning. By enabling compact data representations that are compatible with CNN architectures, this work opens new possibilities for deploying deep learning models in real-world, low-power scenarios.

In summary, Mini-Color NN introduces an effective image preprocessing strategy for reducing the computational cost of CNNs, without severely compromising the quality of visual information. It serves as a practical step toward bringing powerful neural network models to devices with limited resources.

II. Background

Image classification is a foundational task in computer vision that involves assigning a label to an entire image based on patterns learned from a pre-labeled dataset. It relies on two essential components: algorithms that can analyze and learn from pixel data, and high-quality image datasets that provide examples of each class for training and evaluation.

A widely used dataset for image classification research is CIFAR-10, which consists of 60,000 color images at a resolution of 32×32 pixels. The dataset is divided into 10 distinct classes:

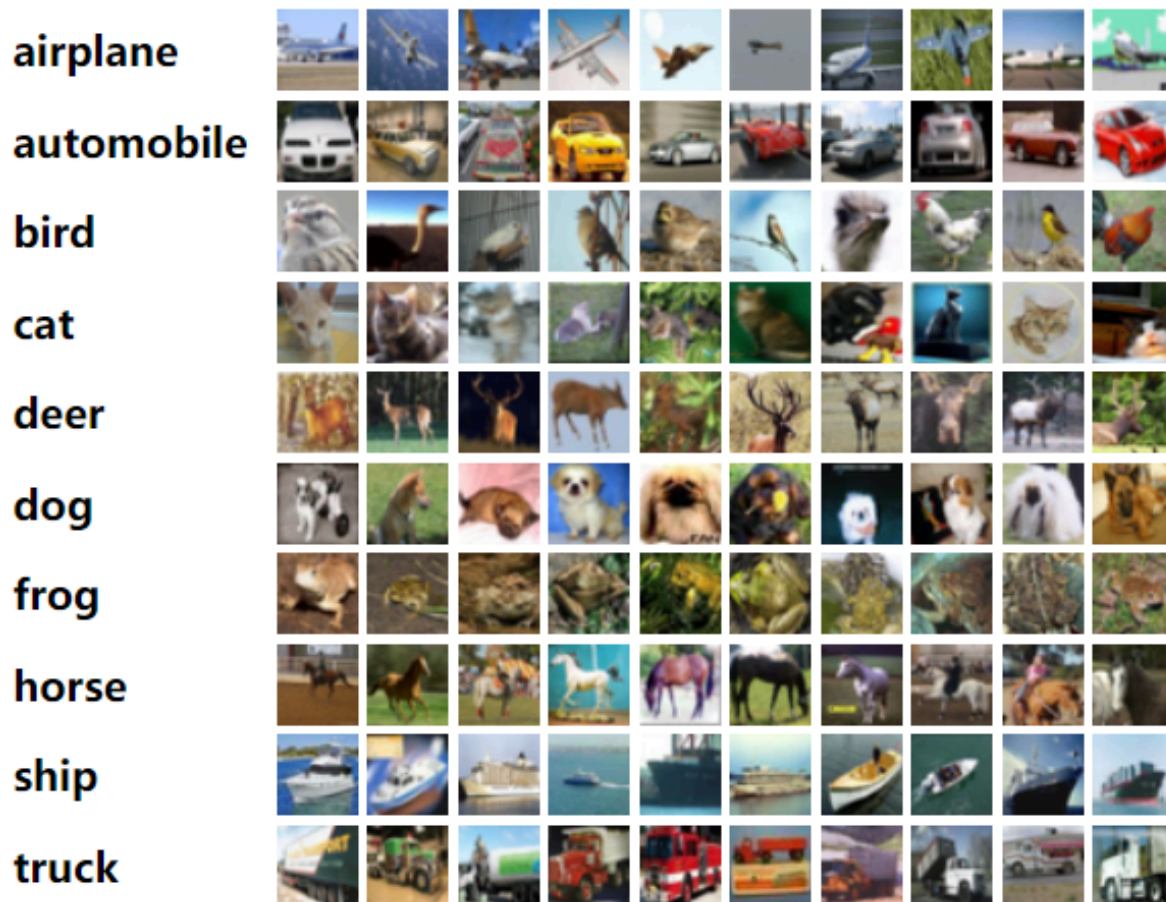


Figure 1. Snapshot of each label in the CIFAR-10 Dataset.

Each class contains 6,000 images, with 50,000 images used for training and the remaining 10,000 reserved for testing. CIFAR-10 was created by the Canadian Institute for Advanced Research (CIFAR) and is a benchmark dataset for evaluating the performance of image classification models.

Each image in CIFAR-10 is represented as a 3D tensor containing height, width, and color channels (red, green, blue). This format allows deep learning models to analyze the spatial structure and color distribution of the image. These models, particularly Convolutional Neural Networks, are well suited for extracting features from image data.

Other solutions to the challenge of running convolutional neural networks (CNNs) efficiently on low-resource devices include innovative approaches like the Dither NN and Quantized CNN. The Dither NN, developed by K. Ando et al. (2018), combines dithering techniques with neural networks to maintain classification accuracy even when operating with low bit-precision hardware. This model was tested on datasets such as MNIST and ImageNet-12 and demonstrated

impressive results, achieving over a $15\times$ compression ratio and $2\times$ acceleration, with less than a 1% drop in accuracy. In fact, the model with dithering outperformed the baseline by reaching 89.14% accuracy compared to 85.83%. Another approach, Quantized CNN, focuses on compressing and accelerating CNNs through quantization methods. This strategy led to a $1.5\times$ speed increase and 2 to $3\times$ data compression, enabling accurate image classification on mobile devices within just one second. Both of these methods demonstrate effective alternatives for optimizing neural networks, particularly in resource-constrained environments, without significantly compromising performance.

III. Applications:

The applications of this color quantization and dithering approach for CNN-based image classification are particularly impactful in environments where computational resources are limited. One primary use is in real-time image processing on low-power sensor cameras, such as those used by firefighters. These cameras often operate in harsh, high-temperature conditions and need to process and classify visual data quickly without relying on bulky hardware. By reducing the color complexity of input images, the system can maintain performance while using less memory and energy.

Another major application is in surveillance systems, where thousands of images or video frames are processed continuously. Applying color quantization and dithering can help reduce the storage space needed for this vast amount of visual data, while also speeding up the classification process. This leads to faster response times and more efficient monitoring, which is essential in security and public safety contexts.

Beyond these, the method is valuable in edge computing devices, such as Internet of Things (IoT) cameras, drones, and mobile robots, where hardware limitations make traditional CNN processing impractical. It also supports research applications by providing a lightweight, publicly available dataset that others can use to experiment with or develop more efficient machine learning models. Overall, this technique enhances the reach and practicality of deep learning models in real-world, resource-constrained environments.

IV. Methods:

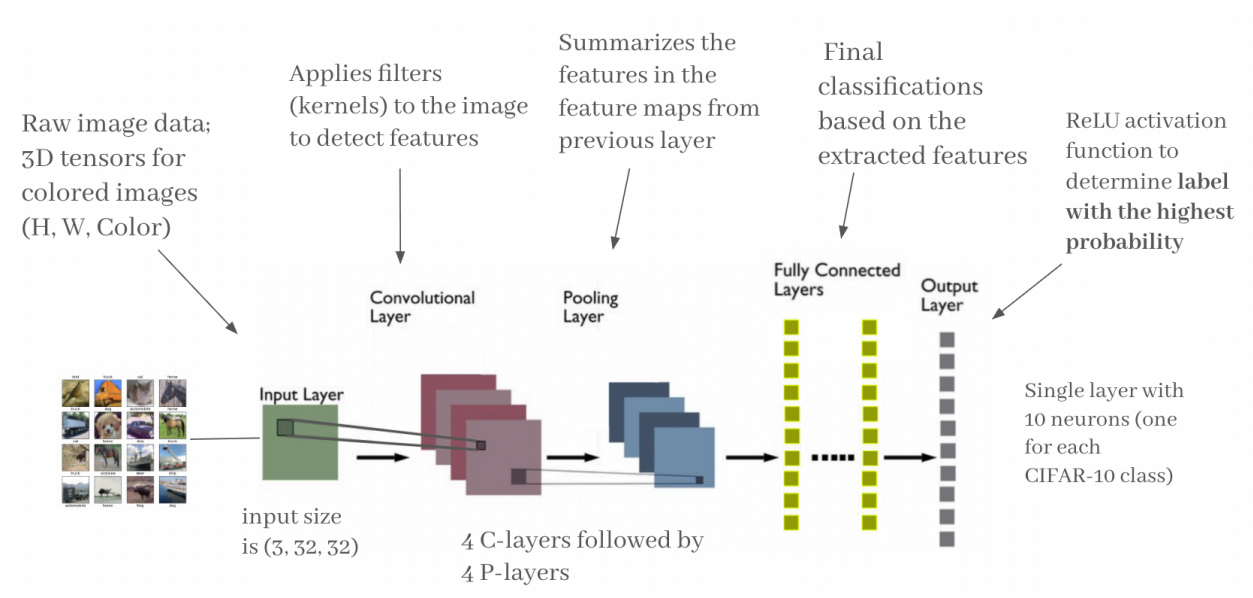


Figure 2. Labelled Convolutional Neural Network Diagram

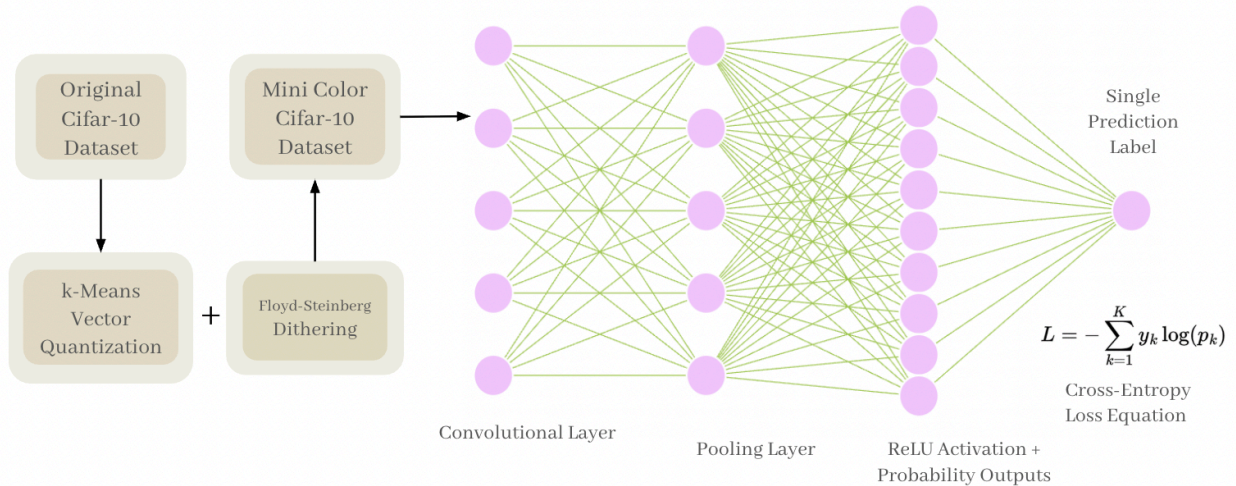


Figure 3. Flowchart of Dataset Modifications and CNN

The input to this project was the CIFAR-10 dataset, which contains 60,000 color images sized 32×32 pixels, distributed across 10 distinct categories. Each image is a 3D tensor consisting of red, green, and blue pixel values. The goal was to create a more computationally efficient version of this dataset that could be used to train a convolutional neural network (CNN) without a significant loss in classification accuracy. To achieve this, the original dataset was processed using two main techniques: k-Means Vector Quantization and Floyd-Steinberg Dithering. The output of this preprocessing was a modified, color-reduced version of CIFAR-10, referred to as the Mini Color CIFAR-10 dataset, which maintained important visual features while significantly reducing pixel-level complexity.

The transformation process began with applying k-Means clustering to each image, treating each pixel's RGB values as a data point in 3D space. The clustering algorithm grouped similar colors into a fixed number of representative color clusters (e.g., 8, 16, or 27 colors). The image was then reconstructed using only these representative colors. However, this quantization introduced visual artifacts. To address this, Floyd-Steinberg Dithering was applied. This technique calculates the difference between the original pixel color and its quantized version (the quantization error), then diffuses this error to neighboring pixels using a specific coefficient matrix (*Figure 4*). This diffusion maintains the overall visual fidelity of the image even with fewer available colors.

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{array}$$

Figure 4. Steinberg Error Diffusion Matrix to Bordering Pixels

Once the images were processed and the Mini Color dataset was created, they were used to train a CNN. The architecture of the CNN, as shown in the diagram, consisted of four convolutional layers followed by four pooling layers, and concluded with a fully connected output layer with 10 neurons, one for each class in CIFAR-10. The CNN took the color-reduced 3D images as input and used filters to extract hierarchical features from the data. These features were pooled to reduce dimensionality, and finally classified using ReLU activation and a softmax layer. The model was trained using cross-entropy loss to measure the error between predicted and true labels.

The motivation for using these methods came from the need to reduce computational demand for image classification on resource-constrained devices like mobile phones, drones, and IoT systems. Reducing the number of unique colors in the image decreases the amount of data the CNN needs to process, making training and inference faster and less memory-intensive. Additionally, using error-diffusion dithering ensures that the quality of the input data remains high, helping the CNN retain accuracy even with compressed input.

Several challenges arose during experimentation. Lower values of k in the quantization step (e.g., k=4 or k=8) made some images nearly unreadable, leading to a drop in classification accuracy. Also, the basic version of k-Means was computationally intensive and slow, especially for larger k values. If more time were available, the implementation would be improved by integrating k-Means++ to speed up clustering and by fine-tuning the neural network to better

handle the quantized data. Additional improvements could include testing other dithering methods for even better visual quality and expanding the experiment to larger datasets like CIFAR-100 or ImageNet.

If the project were to be redone, real-time performance evaluation would be a key focus, specifically testing the model on actual low-resource hardware to directly measure speedups and memory savings. Furthermore, building a dataset-specific CNN architecture tailored to the reduced color input could enhance efficiency even further.

V. Results:



Figure 3. Comparison of Original, Dithered 8 colors, and Dithered 27 colors

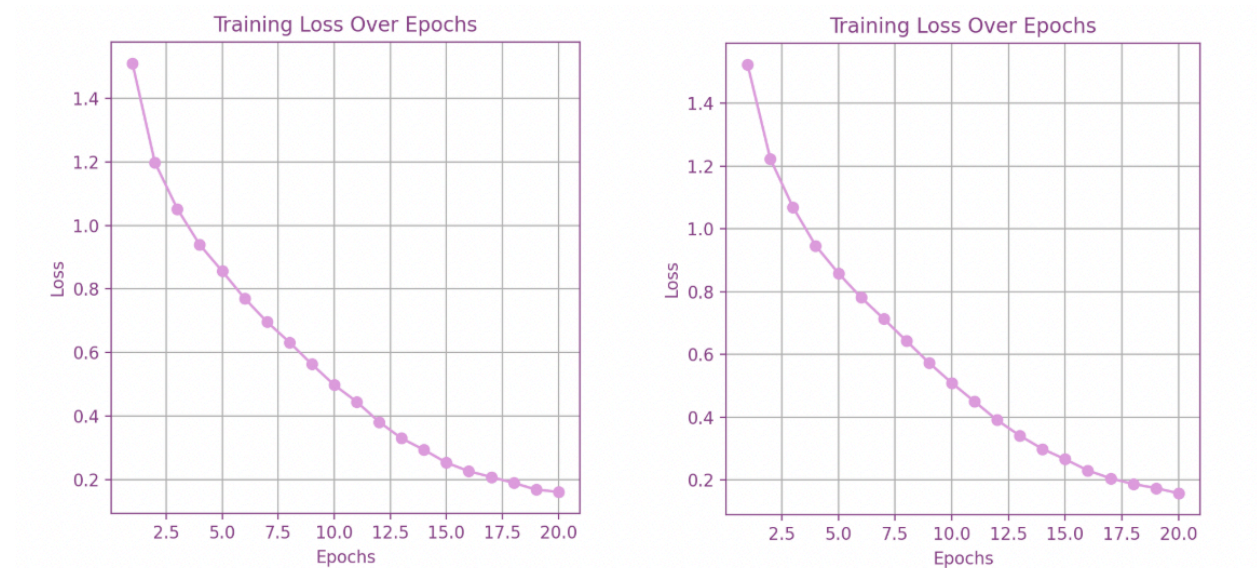


Figure 4. Comparison of CNN Loss Graphs. Original (L) vs. Mini Color (R)

The results of the project demonstrated that applying color quantization and dithering techniques to the CIFAR-10 dataset led to notable improvements in efficiency while maintaining high classification accuracy. When the CNN was trained on the original CIFAR-10 dataset, it achieved an accuracy of 92.88% after approximately 50 epochs, with a cross-entropy loss of 0.0772 and an average training time of 78.34 seconds per epoch. In comparison, the modified dataset, referred to as the Mini Color CIFAR-10, produced an accuracy of 92.01%, with a slightly higher cross-entropy loss of 0.0927. However, the training time dropped significantly to 61.22 seconds per epoch, indicating a more computationally efficient process.

The compression ratio of the color-reduced dataset was approximately 1.7 $\times$, as the storage requirement was reduced from 186.2 MB to 105.2 MB, and the acceleration ratio (in terms of training speed) was approximately 1.3 $\times$. These improvements highlight that reducing color complexity through vector quantization and Floyd-Steinberg dithering can make convolutional neural networks more suitable for low-resource environments, with minimal impact on accuracy. The results support the hypothesis that simplified image inputs can still be effectively processed by CNNs, making this approach promising for real-world applications like mobile vision systems and embedded AI devices.

VI. Limitations:

Despite its promising results, the project faced several limitations. One key limitation was that aggressive color quantization, especially with very low values of k (e.g., 4 or 8), significantly degraded image quality, making it harder for the CNN to recognize certain features and leading to reduced classification accuracy. Additionally, the basic k -Means algorithm used for quantization was computationally expensive, particularly with higher k values and across large datasets, resulting in long preprocessing times. The approach also assumes that all images benefit equally from color reduction, which may not hold true for more complex or high-resolution datasets. Furthermore, while the CNN performed well on the modified dataset, the model architecture was not specifically optimized for the reduced color input, which may have limited potential performance gains. Real-time deployment and hardware-based testing were also not conducted, leaving some uncertainty about the actual impact on embedded or edge devices. These limitations suggest that further tuning and evaluation are necessary to fully realize the benefits of this method in practical applications.

VII. Conclusion:

In conclusion, this project demonstrated that applying color quantization using k -Means clustering and Floyd-Steinberg dithering to the CIFAR-10 dataset can significantly improve the

efficiency of convolutional neural networks without a major loss in accuracy. By reducing the number of unique colors in each image, the dataset became more lightweight, resulting in faster training times and lower storage requirements. The modified Mini Color CIFAR-10 dataset achieved a classification accuracy of 92.01%, closely matching the original dataset's performance while offering a compression ratio of 1.7× and an acceleration ratio of 1.3×. These results support the hypothesis that reducing pixel-level complexity can make CNNs more suitable for resource-constrained environments like mobile devices and IoT systems. Although limitations such as quantization time and image quality degradation at low color depths were encountered, the project provides a strong foundation for future exploration into efficient AI processing. With further optimization and real-world testing, this approach has the potential to enhance deep learning applications in edge computing and low-power systems.

VIII. Future Work:

Future work will focus on improving the quantization process by exploring more efficient techniques such as K-means++ to reduce information loss and enhance computational speed. Additionally, fine-tuning the neural network weights to better adapt to quantized images could improve overall image quality and readability. Addressing challenges with images containing many colors will also be important, as preserving visual fidelity in such cases remains difficult. These efforts aim to make the method more robust and effective across a wider range of image types.

IX. Materials:

This project was fully executed and stored on an Apple MacBook Pro (14-inch, 2021) running macOS Monterey, equipped with an Apple M1 Pro chip.

Original Python version of CIFAR-10 Dataset obtained from Alex Krizhevsky at <https://www.cs.toronto.edu/~kriz/cifar.html>

References:

1. K. Ando et al. (2017). *Color image quantization using adaptive spatial color indexing*. IEEE Transactions on Consumer Electronics, **63**(3), 264–271. <https://doi.org/10.1109/TCE.2017.8171208>
2. Balasubramanian, V. N., Venkatesh, R., & Sheorey, T. (2019). *Deep color quantization*. In 2019 IEEE Winter Conference on Applications of Computer Vision (WACV) (pp. 1697–1705). IEEE. <https://doi.org/10.1109/WACV.2019.00185>

3. Moroney, N. (2023). *cq100: A high-quality image dataset for color quantization*. *Journal of Electronic Imaging*, 32(3), 033019. <https://doi.org/10.1117/1.JEI.32.3.033019>
4. SciPython. (n.d.). *Floyd–Steinberg dithering*. SciPython.com. Retrieved May 20, 2025, from <https://scipython.com/blog/floyd-steinberg-dithering/>
5. SuperAnnotate. (n.d.). *Image classification: The basics*. SuperAnnotate Blog. Retrieved May 20, 2025, from <https://www.superannotate.com/blog/image-classification-basics>
6. Vitalflux. (n.d.). *Different types of CNN architectures explained with examples*. Vitalflux.com. Retrieved May 20, 2025, from <https://vitalflux.com/different-types-of-cnn-architectures-explained-examples>

The project utilized several Python libraries essential for deep learning and data processing. PyTorch served as the core framework, with `torch` providing the fundamental tensor operations and model management. The `torch.nn` module was used to define the neural network architecture, while `torch.optim` provided optimization algorithms like SGD and Adam for training. Functional operations such as activation functions and loss calculations were handled through `torch.nn.functional`. To preprocess and transform image data, the project employed `torchvision.transforms`. Data loading and batching were managed using `DataLoader` and `TensorDataset` from `torch.utils.data`, enabling efficient handling of training and testing datasets. Visualization of images and training progress was done with `matplotlib.pyplot`. Additionally, `numpy` was used for numerical operations, and `pickle` was included for saving and loading Python objects such as datasets or models. The `os` module facilitated file system interactions, and `perf_counter` from the `time` library was used to measure the performance and execution time of various operations, particularly useful for benchmarking quantization and training efficiency.

APPENDIX:

- I. CODE: <https://github.com/disco-io/SysLab-RES-24-25>
- II.